

Which controller should steer a survey boat?

PID, line-of-sight and model-predictive control compared on the water

36 runs: 4 routes $\times$ 3 controllers, flown in real water and repeated twice in simulation.

Abstract

Three ways of making a BlueBoat follow a planned line were flown back to back over four routes, then repeated in a simulator with and without a current. Model-predictive control stayed on average **0.29 m** from the planned line against 0.43 m for PID and 0.40 m for line-of-sight; more tellingly, its worst deviation anywhere was **1.20 m** against **4.79 m** for PID. It alone finished at the speed the route was planned for, and paid for that with half as much thrust again per metre. Every number is recomputed from the logs by `analyze.py`, re-checked by `verify.py`, and listed per run in `metrics_all.csv`.

1 How the three controllers work

The boat has one thruster on each side: pushing both equally drives it forward, pushing one harder than the other turns it. Whatever the method, a controller only ever decides two numbers — the force asked of each thruster — twenty times a second, and those forces are limited to ± 20 newtons. If a controller asks for more, **both** are scaled down by the same factor rather than cut individually, so the boat still turns the way it intended, just less forcefully.

None of the three follows a stopwatch. The route is a line drawn on the water with a speed written along it, and a **target point slides forward along that line**. It only advances while the boat is keeping up: more than 3 m behind and the point stops and waits; between 0.5 m and 3 m behind it moves at a reduced speed; within 0.5 m it moves at the full planned speed. This rule is the **governor**, and it is shared by all three controllers. A boat can therefore never be abandoned by its own reference — but one that habitually trails is handed a permanently slower reference, and the whole survey takes longer. That turns out to be the largest practical difference between the three.

Table 1: The three controllers under comparison: what each one looks at on the route, its governing idea, and the gains actually flown on the physical boat. Gains are the defaults in `master_control.py` for `isSimulation == False`.

	What it looks for on the route	Main idea of the method	Tuning used on the boat
PID	a point 2.5 m further along the route than the nearest point, and how far ahead or behind the boat is along the route	Two nested stages. The outer pair turns position and heading error into a demanded speed and turn rate; the inner pair drives the boat onto those demands and sets the thruster forces. Every loop keeps a running total of past error, so a steady push from wind is eventually cancelled — but nothing caps that total.	Aim-ahead $\Delta = 2.5$ m. <i>Outer</i> x — along-track error $\rightarrow$ speed demand: $k_p 3.0, k_i 0.01, k_d 0$. <i>Outer</i> ψ — heading error $\rightarrow$ turn-rate demand: $k_p 3.0, k_i 0.01, k_d 0$. <i>Inner</i> u — speed error $\rightarrow$ forward force: $k_p 1.0$. <i>Inner</i> r — turn-rate error $\rightarrow$ turning moment: $k_p 1.5$.
Line-of-sight	the same point 2.5 m further along the route, and the speed written on the route there	One stage, no memory of past error. It steers at the aim-ahead point and asks for the route's own speed, never for a speed that would close a gap — so once behind the target point it matches that point's speed instead of catching up, and settles into a fixed trailing distance.	Aim-ahead $\Delta = 2.5$ m — the further off the line, the harder it aims back. $k_u = 20$ — forward force per unit of speed error (N per m/s). $k_\psi = 10$ — turning moment per unit of heading error. $k_d = 1.0$ — turning moment opposing the present turn rate, i.e. damping. Speed scale = 1.0 — the route's own speed is passed through unchanged.
Model-predictive	the whole next 2.5 s of route, in fifteen steps	It carries a model of how this hull responds to thrust. Every 50 ms it searches for the force sequence that keeps the boat closest to that stretch of route for the least thrust, applies only the first step, then solves again. The force limit sits inside the search, and planning ahead lets it turn <i>before</i> a corner.	Horizon 15 steps over 2.5 s, one step every 0.167 s. State weights $Q = \text{diag}(50, 50, 30, 1, 1, 1)$ on (x, y, ψ, u, v, r) — position and heading count far above the velocities. Effort weights $R = \text{diag}(0.015, 0.015)$ on the two thruster forces, as an absolute penalty with no rate term. Hard input bounds ± 20 N inside the solver. Working-set budget 120 iterations, $\max(50, 4n_u N)$: the active-set solver needs at least as many iterations as the condensed problem has variables, or a saturating solve fails outright. Plant model 16.01 kg, 5.64 kg m ² in yaw, with added-mass and damping coefficients fitted to the hull.

The simulator was run with different tuning for all three controllers. Those values are not listed here, because Section 5 uses simulation only to confirm the ordering of the methods, not their exact numbers.

2 The four routes

Table 2: The four planned routes flown by every controller, and the failure mode each one is designed to expose.

Route	What it looks like	Planned	What it is meant to expose
Sine weave	S-shaped weave, 9 m across, three cycles	132.2 m at 0.5 m/s	continuously reversing curvature: are the bends cut?
Expanding square	square spiral, first side 4 m, +7 m each side	174.5 m at 0.5 m/s	repeated sharp 90° corners: are they overshoot?
Tough angles	straight leg, 150° arc, long curved sweep	155.6 m at 0.7 m/s	abrupt <i>changes</i> of curvature, at the fastest speed
Lawnmower	20 × 40 m box, six passes 7 m apart	228.1 m at 0.5 m/s	the realistic survey: straights, 180° turns, endurance

3 Where and when the runs were made

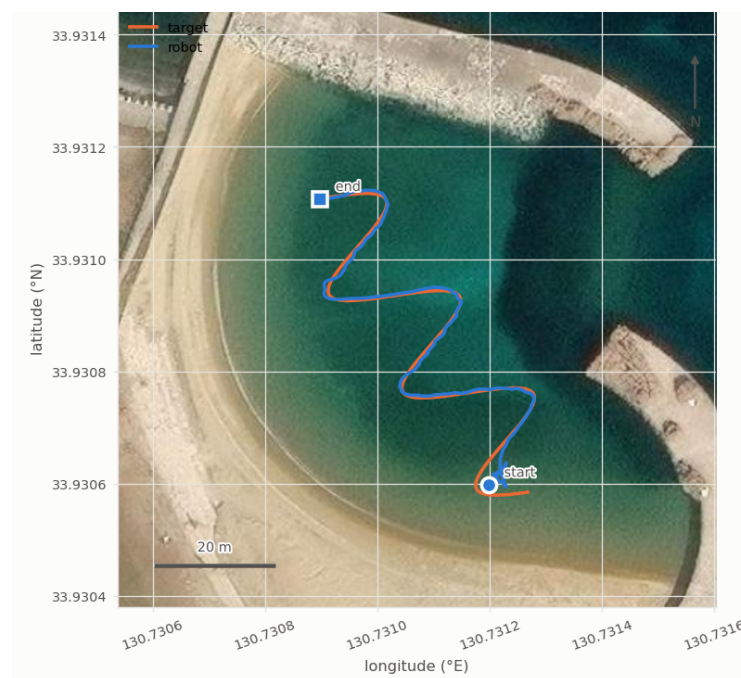


Figure 1: The field site. The PID run of the sine weave over satellite imagery. The basin opens at its **north-east corner**; wind and flow pushed the boat westward all day.

The twelve field runs were flown back to back between **10:39 and 12:26**, route by route (sine weave, expanding square, tough angles, lawnmower) and within each route PID, then line-of-sight, then model-predictive. The wind blew from the north / north-east all morning: the Kitakyushu shore record gives a mean of 21 km/h at 09h, 14 km/h at 12h and 27 km/h at 15h, gusting to 35, 22 and 33 km/h. The basin is semi-enclosed, so waves were smaller than open sea, but they visibly moved the boat.

This is the main weakness of the comparison. The flow inside the basin cannot be recovered from a

shore wind reading, and the wind roughly halved and then doubled again across the two hours the runs took, so no two runs met the same conditions. Differences smaller than the scatter between runs cannot be resolved by this data.

Two things about the boat, in the operator's own words:

"In our boat, the right thruster appeared weaker than the left one, but no compensation gain was used during the field experiments nor simulation, as nothing showed a need for it. Also, before sending commands to the robot, in case of saturation of the input (exceeding the $\pm 20\text{ N}$ limits), a proportional scaling was applied to preserve direction."

Both are confirmed in the flown code: the correction factor is written, then immediately overwritten with 1.0.

The recordings show that imbalance plainly. The sine weave and the lawnmower turn left as often as right, so on those two a balanced boat should ask both thrusters for the same average force. The real boat asked the **right** thruster for **2.3 to 3.3 newtons more** than the left; the same two routes in the simulator, where the thrusters are identical, come out between -0.4 and $+0.3$ newtons. The asymmetry is in the boat, and all three controllers were absorbing it.

4 What happened in real water

Two distances appear below and they do **not** mean the same thing. **Distance from the route** — also called the tracking error — is measured sideways, at right angles to the planned line: it says whether the boat actually passed over the ground the route was drawn across. **Trailing distance** is how far behind its moving target point the boat sits; a boat can be exactly on the line and still 1.5 m from the target point simply because it is behind it. Trailing costs time, not coverage.

Control effort is the total commanded thruster force, added up over the run and divided by the metres of route covered, so a slow run is not rewarded for being slow. **Accuracy bought with effort** is not a single number: it is those two quantities read as a pair, as in Figure 3(c) — average distance from the route on the vertical axis, control effort on the horizontal. Low on the vertical axis means the boat held the line; far to the right means it burned force to do so. A controller that sits down *and* to the left is better outright; one that gets down only by moving right is asking the operator to pay for its accuracy in battery and thruster wear.

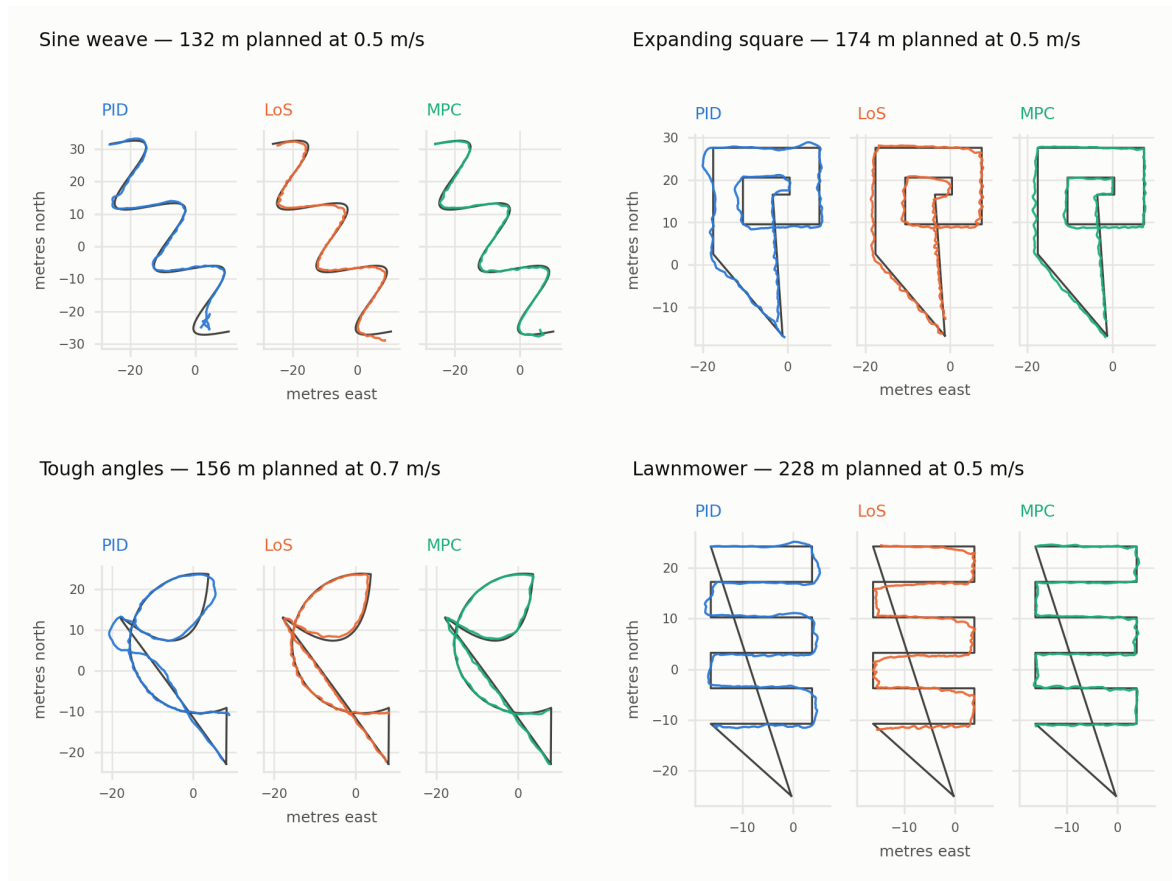


Figure 2: Planned route against measured track. All twelve field runs, at true 1:1 scale: four routes, and within each, one small panel per controller. The thin dark line is the planned route; the coloured line is where the boat actually went. PID's loop at the start of the sine weave, its overshoot at the corners of the expanding square and its wide bulge on the 150° arc of the tough-angles route can be read directly.

Tracking figures start from the moment the boat first came within 1 m of the route, so the time spent getting onto the line does not contaminate them; how long that took is reported in Section 7.

Table 3: The twelve field runs. *Average* and *worst* are the sideways distance from the planned line, in metres. *Over 1 m* is the share of the run spent further than a metre from it. *Speed against plan* is progress along the route divided by the speed it was drawn at, so 100% means the survey ran to schedule. *Thrust per metre* is newton-seconds of commanded thrust per metre covered. Best of the three on each route in bold.

Controller	Average (m)	Worst (m)	Over 1 m	Speed against plan	Thrust per metre	Covered	Time
<i>Sine weave</i>							
PID	0.33	0.98	0.0%	79%	15.2	98%	332 s
Line-of-sight	0.34	1.37	2.9%	66%	16.5	99%	393 s
Model-predictive	0.25	0.77	0.0%	99%	22.0	99%	265 s
<i>Expanding square</i>							
PID	0.49	2.49	11.4%	88%	15.5	98%	389 s
Line-of-sight	0.47	1.30	4.1%	65%	17.6	97%	523 s
Model-predictive	0.41	1.20	1.3%	99%	25.5	98%	346 s
<i>Tough angles</i>							
PID	0.46	4.79	9.7%	74%	14.8	90%	272 s
Line-of-sight	0.32	1.03	0.4%	65%	13.1	91%	312 s
Model-predictive	0.27	0.80	0.0%	100%	19.8	92%	204 s
<i>Lawnmower</i>							
PID	0.42	1.54	4.0%	85%	16.3	68%	365 s
Line-of-sight	0.47	1.47	5.9%	62%	19.6	68%	492 s
Model-predictive	0.24	0.63	0.0%	99%	30.2	68%	313 s

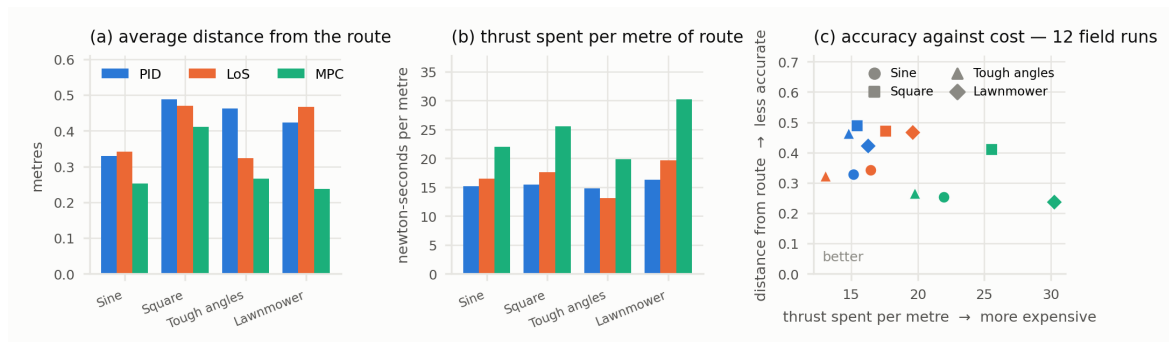


Figure 3: Accuracy, cost, and the trade between them. Field runs only. (a) average distance from the planned route and (b) thrust spent per metre, by route and controller. (c) the same twelve runs with the two against each other: lower is closer to the route, further right is more force spent, so the bottom-left corner is the best place to be.

Why two of the three finish late. The governor is the reason. A controller that sits well behind its target point is in effect telling the governor to slow down, and the governor obliges for the rest of the run. Model-predictive control never falls behind — it sits a fraction of a metre *in front* of its target point — so it is never throttled and covers the route at the speed the route was drawn for. PID sits 0.7 to 1.2 m back and gets 74–88% of that speed. Line-of-sight sits 1.3 to 1.4 m back, by design rather than by accident, and gets 62–66%. The link is tight enough to check directly: take each run’s measured trailing distance, put it into the governor’s rule, and the speed it predicts matches the speed the run actually achieved to within 5%, in all twelve field runs. The cost shows up at the end of the day rather than in the track — the same four routes took **1128 s with model-predictive control, 1358 s with PID (20% longer) and 1721 s with line-of-sight (53% longer)**.

The boat was being shaken, and the controllers were not doing it. In the field the hull rolled with

a standard deviation of $1.6\text{--}2.6^\circ$ and its roll rate averaged $0.15\text{--}0.24\text{ rad/s}$; in both simulations, which contain no waves, those figures are $0.13\text{--}0.28^\circ$ and 0.003 rad/s — sixty times smaller, with a clear field rhythm near one cycle per second that is short wind chop rather than swell. The boat also reversed its turning direction 7 to 26 times a minute in the field against 0.2 to 5.4 in simulation, for all three controllers alike, so those reversals are the sea moving the boat, not a controller hunting.



Figure 4: One route, second by second. The sine weave in real water. Top: distance from the planned route. Bottom: speed, each run’s average dashed in its own colour, the planned 0.5 m/s in grey.

5 Does the simulator agree?

The twelve missions were repeated twice in Gazebo: once in calm water, once with a current of about 0.1 m/s running north-north-east to south-south-west, worth roughly 2.5 N of drag on a stationary boat.

Table 4: Field and simulated performance, averaged over the four routes. Best of the three controllers in each row in bold.

Averaged over the four routes	PID	Line-of-sight	Model-predictive
Average distance from the route, real water (m)	0.43	0.40	0.29
... in calm simulation	0.18	0.07	0.07
... in simulation with a current	0.39	0.42	0.18
Thrust per metre, real water	15.4	16.7	24.4
... in calm simulation	41.1	38.4	43.6
... in simulation with a current	44.0	41.8	46.6
Speed against plan, real water	81%	64%	99%
... in calm simulation	78%	70%	99%
... in simulation with a current	75%	67%	94%

Four things carry across. The **ordering holds** in all three conditions: model-predictive control the most accurate and most expensive, line-of-sight the cheapest. The **speed penalty transfers exactly** — each controller’s share of the planned speed moves by at most 0.07 between real water and either simulation, confirming it comes from the governor, not from the weather. **Calm simulation is far too optimistic**, by two to six times; adding the current brings the errors back into the measured range, hurting line-of-sight most ($0.07 \rightarrow 0.42\text{ m}$) and model-predictive control least ($0.07 \rightarrow 0.18\text{ m}$). And **PID’s weakness is the controller, not the weather**: in calm simulation, with no wind and no waves at all, it still goes **4.76 m** off the tough-angles route — within three centimetres of the 4.79 m it managed in the field — and it is the only one of the three that ever exceeds a metre in calm water.

One caveat in PID's favour: its simulated gains were never re-tuned for the simulated hull, so part of its poor showing in simulation is tuning rather than method, and a proper re-tune there would likely close some of that gap. It would not close the part that also appears in the field, where the gains *were* tuned for the real boat.

6 What to choose

Table 5: Recommended controller by route, with a fallback where the force budget is tight.

Route	Recommended	Fallback if the force budget is tight	Avoid
Sine weave	model-predictive	PID: as accurate as line-of-sight here, and 15% faster	—
Expanding square	model-predictive	line-of-sight: PID overshoots the corners	PID
Tough angles	model-predictive	line-of-sight: cheapest of all twelve runs, within 0.05 m of the best	PID
Lawnmower	model-predictive	PID: cheaper and more accurate here than line-of-sight	—

What this comparison cannot tell you. One site, one day, one run per combination, with the wind changing by a factor of two across the two hours — so any difference smaller than the scatter between runs is not resolvable here. The simulator used different gains and hull model, so it supports the ordering of the methods and nothing finer. Positions were logged three times a second, making the command-activity figures a lower bound. The three lawnmower runs were all stopped at the same 68% of the route: comparable with one another, but none is a completed survey.

7 Detailed conclusion

Choose model-predictive control wherever the boat's power and thruster budget allow it. The case is not marginal. It was the most accurate on every route (0.24–0.41 m average, against 0.33–0.49 m for PID and 0.32–0.47 m for line-of-sight); its worst moment anywhere was 1.20 m against PID's 4.79 m; it reached the line within 0.7 s of every start; and it is the only controller that never spent a measurable part of a run more than a metre off the line. How much that matters depends on what the boat is carrying, and coverage is the clearest example: the lawnmower's passes were set 7 m apart so that the swaths of the **side-scan sonar mounted on the USV** overlap from one pass to the next. A controller whose bad moments stay near 0.5 m keeps that overlap intact, whereas PID's single 4.79 m excursion is more than half a line spacing — a strip of ground left unimaged rather than a cosmetic blemish. And because it alone holds the planned speed, the same four routes finished **20% sooner than with PID and 53% sooner than with line-of-sight**, which on a weather-limited field day converts directly into more lines covered.

What it costs is wear and power, not accuracy. Model-predictive control spends 24.4 newton-seconds of thrust per metre against 15.4 for PID and 16.7 for line-of-sight, commands roughly twice the total thrust, changes its command about thirteen times more often than PID, and sits at the ± 20 N limit for 7.8% of an average run and 13.9% of the lawnmower. It is the only one working near the limits of the thrusters, and the only one whose cost climbs steeply with how much turning a route demands — 19.8 newton-seconds per metre on the tough-angles route against 30.2 on the lawnmower. It also needs an optimisation solved every 50 ms where the other two need a few lines of arithmetic, which is a real constraint if the controller must ever move to a smaller on-board computer.

Where endurance, thruster wear or a tight force budget dominates, use line-of-sight. It is the

cheapest and the most even-tempered of the three: it produced no large excursion on any route, and on the tough-angles route it was at once the cheapest run of the campaign and within 0.05 m of the best accuracy measured. Its price is explicit and permanent, and it is a property of the method rather than of its tuning — with no memory of past error and a forward speed copied from the route instead of computed to close a gap, it settles 1.3–1.4 m behind its target point, and the governor turns that into a mission roughly a third longer. If the survey is limited by daylight or weather rather than by battery, that is the wrong trade.

Use PID only on gentle routes. It is the cheapest on three routes out of four and the better classical method on the sine weave, where it is as accurate as line-of-sight and 15% faster. But it is the only controller that produced deviations beyond 2 m, and it did so on two different routes; on the expanding square it spent 11.4% of the run more than a metre off the line, overshooting the corners. It is also the least dependable at the start: the operator’s note on the sine weave describes exactly what the top panel of Figure 4 shows —

“Notice that PID struggled at the beginning: it made the robot going backward + rotating before finally reaching the path”

— and it took 50 s to get onto the line, against 0.7 s for model-predictive control. This happens intermittently with this controller and is consistent with its running totals of past error, which have no ceiling and can build up while the boat is still far away. Calm simulation reproduces its worst field excursion to within three centimetres, so better weather will not fix it.